



南京大學

本科畢業論文

學院 智能軟件與工程學院

專業 軟件工程（智能化軟件）

題目 面向實踐的緊湊哈希表設計與
簡化復現

年級 2022 學 號 221900323

學生姓名 紀澤操

指導教師 劉明謀 職 稱 副教授

提交日期 2026 年 5 月 30 日



南京大学本科毕业论文（设计） 诚信承诺书

本人郑重承诺：所呈交的毕业论文（设计）（题目：面向实践的紧凑哈希表设计与简化复现）是在指导教师的指导下严格按照学校和学院有关规定由本人独立完成的。本毕业论文（设计）中引用他人观点及参考资源的内容均已标注引用，如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文（设计）等违纪行为。

作者签名：纪泽操

学号：221900323

日期：2026 年 5 月 30 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：面向实践的紧凑哈希表设计与简化复现

学院：智能软件与工程学院

专业：软件工程（智能化软件）

本科生姓名：纪泽操

指导教师（姓名、职称）：刘明谋 副教授

摘要：

本论文以《On the Optimal Time/Space Trade off for Hash Tables》为实验方向，系统整理文中提及的理论算法及核心思想，然后加以简化并落实到实际当中，从宏观架构层面，搭建了 Facility - Cubby - Slot 三层存储体系，从而达成高效地组织和运作哈希表的目的；微观机制层面，融合了可逆哈希函数，局部查询路径，商压缩，K - Kick 规则，多层级 Cubby 以及双表渐进式扩缩容迁移等主要技术。落地的哈希系统包含初始化，插入，删除，查询，扩容以及性能指标验证等完整的操作，实验结果表明，该系统可以稳定执行各类动态操作，其路由访问步数，元素被踢出的移动次数以及元数据编码位数等关键指标均处于可控制范围内，能够精准再现原始理论架构的核心逻辑，而且，本文明确了工程简化达成与严格理论模型之间的差异界限，给类似紧凑哈希结构的工程化重现及其在实际场景中的应用赋予了有价值的参考。

关键词：哈希表；时间空间权衡；探测复杂度；增强开放寻址；k-kick 树；动态扩容

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Practical Compact Hash Table Design and Simplified Reproduction

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

This thesis is built on the study of the paper *On the Optimal Time/Space Tradeoff for Hash Tables*. It does an analysis on the theoretical algorithms and main ideas in the paper, and then tries to make them work in real engineering by using a more simple and practical way. From the overall structure point of view, a three-layer storage model made up of Facility, Cubby, and Slot is designed, which helps to manage the hash table in a more organized way. At the implementation side, the system uses many key techniques like reversible hash functions, localized query paths, quotient compression, the K-Kick rule, multi-tier Cubby organization, and incremental dual-table resizing and migration. The built hash system can support all basic functions such as initialization, insertion, deletion, lookup, dynamic resizing, and performance testing. The experiment results show that it can handle different dynamic operations stably, and important indicators like routing probe counts, kick displacement frequency, and metadata encoding overhead are kept in acceptable levels. The implementation is able to get the core behavior of the original theoretical design in a reasonable way. Also, this work makes a discussion about the differences between the engineering simplifications used here and the pure theoretical model, which can be helpful for the practical rebuilding and use of compact hash table structures.

KEYWORDS: hash tables; time–space tradeoff; the Probe-Complexity problem; enhanced open addressing; k-kick tree; dynamic expansion

目 录

目 录	III
第一章 引言	1
1.1 实验背景及意义	1
1.2 系统结构概要	2
1.3 国内外现状	2
1.4 实验路线	3
第二章 问题模型与理论基础	4
2.1 探测复杂度与球槽分配模型	4
2.2 系统核心设定：全域与哈希基础	5
2.2.1 全局参数	5
2.2.2 可逆置换与全局路由哈希	5
2.2.3 Facility 编号与 Cubby 内局部地址	6
2.3 核心组件层级关系	6
2.4 商压缩的信息论思想	7
2.5 k-kick、MiniArray 与 Local Query Router 的理论角色	8
2.5.1 k-kick：逻辑分层与交换上界	8
2.5.2 MiniArray M ：空闲 bin 的常数时间判定	9
2.5.3 Local Query Router： $O(1)$ 查询与映射存储	9
2.5.4 辅助结构同步不变量	9
2.6 本章小结	10
第三章 哈希表结构与算法设计	11
3.1 Facility 与 MiniArray	11

3.1.1	Facility 的基本属性	11
3.1.2	MiniArray 的逻辑结构	11
3.1.3	Access: 按逻辑下标读取	12
3.1.4	Insert: 插入与叶分裂	12
3.1.5	Delete: 删除与叶合并	13
3.1.6	Update: 原地覆写	14
3.1.7	Split 与 Merge	14
3.1.8	Rank 与 Select	14
3.1.9	复杂度与空间性质	15
3.2	多层次 Cubby 设计	15
3.2.1	层级属性与初始状态	15
3.2.2	动态调整: 合并与拆分	16
3.2.3	Cubby 内部组件	16
3.2.4	插入与删除操作	17
3.3	k-kick 层级插入算法	17
3.3.1	层级 bin 定义	18
3.3.2	偏好序列	18
3.3.3	探测位置序列	19
3.3.4	bin 饱和与最大可用深度	19
3.3.5	随机深度选择与 InsertKick	20
3.3.6	ReInsert: 被踢元素向上放入	21
3.3.7	复杂度与正确性	21
3.4	Local Query Router	22
3.4.1	设计目标	22
3.4.2	映射规则	23
3.4.3	二叉前缀树存储与接口	23
3.4.4	复杂度与 Patricia 压缩	23
3.5	商压缩与键恢复	25
3.5.1	设计动机与信息分工	25
3.5.2	$\pi(x)$ 与 $g^*(x)$ 的位段划分	25

3.5.3	MiniArray B 中的 MetaEntry	26
3.5.4	恢复: 重建 $\pi(x)$	27
3.5.5	查询校验	27
3.5.6	空间性质	28
3.6	插入、查询与删除	28
3.7	本章小结	29
第四章	系统性能实验	30
4.1	实验目的与环境	30
4.2	实验方案	30
4.2.1	实验分组与工作量	30
4.2.2	评价指标	31
4.3	规模 n 实验	31
4.4	Facility 规模 K 实验	32
4.5	k-kick 深度 k 实验	32
4.6	多层级 Cubby 动态重构实验	33
4.7	元数据开销实验	33
第五章	总结与展望	35
5.1	工作总结	35
5.2	不足与局限	36
5.3	展望	36
	参考文献	37
	致 谢	41

第一章 引言

1.1 实验背景及意义

哈希表是一种通过哈希函数把关键字映射到数组相应位置的结构^[1-2]，它在数据库索引、键值存储、编译器符号表以及运行时系统等常见基础架构中被广泛应用。相较于链地址法，开放寻址哈希表将元素直接放置在连续数组中，其查询路径相对较短，缓存局部性表现更优；但是当负载因子上升时，冲突及探测长度会快速影响操作成本。在实际工程操作中，常采用预留空槽的方式以降低冲突发生几率，但这也会致使单位元素所占据的附加空间有所增加。因此哈希表的设计需解决如下具体问题：在确保查询、插入及删除操作时间可有效控制的基础上，如何减少每个元素为实现定位、更新与恢复而额外存储的信息量。

Bender 等人等人的论文^[3]从理论层面阐述了上述问题：在只能花费常数时间执行查询的前提下，若插入和删除允许承担 $O(k)$ 次元素交换代价¹，则每个元素所占的额外空间可从 $O(\log \log n)$ 比特降至 $O(\log^{(k)} n)$ 比特²。该结果表明，开放寻址哈希表的空间利用率不仅取决于空位占比，还与元素搬移、局部引导及键重建信息的编码方式密切相关。此前，紧凑哈希表在常数时间操作下每键约需 $O(\log \log n)$ 比特冗余；动态检索与 filter 的下界^[4]、Iceberg 系列结果^[5-6] 以及紧凑检索结构^[7] 等已逐步逼近信息论下界。本文在该理论框架下给出完整实现与性能评估：Facility 级 MiniArray、k-kick 层级插入与 Local Query Router 均按原论文语义落地；具体架构与模块分工见下一节，各模块的算法细节留待第二、三章展开。

1 一次更新过程中被搬移（踢出或回填）的元素个数。

2 本文空间开销均按比特计； $\log^{(k)} n$ 表示对 n 连续取 k 次对数。

1.2 系统结构概要

为避免后文术语堆砌以及便于理解，本节仅从宏观上勾勒本文实现所采用的分层组织；各模块的算法语义与代码路径不在此赘述。

从逻辑结构上看，我们可以将整张表视为一个规模为 N 的开放寻址结构。实现上按地址将其划分为若干 Facility（表级分区），每个 Facility 内包含若干 Cubby（局部存储块）及配套的 MiniArray 元数据（用于存储数据长度不同的结构）。元素最终落在 Cubby 的槽位上：槽位只保存经商压缩剥离寻址低位后的载荷比特；恢复完整键所需的变长信息由 Facility 级 MiniArray 与 Cubby 内 $MiniArray(B)$ 另行编码。插入操作会在 Cubby 内沿 k-kick 规则做有界迁移，并由 MiniArray M 维护 bin（k-kick 树种将不同层级的区域称为 bin）空闲状态；查询时 Local Query Router（ $A[i]$ 前缀树）直接给出探测下标，无需扫描完整的探测序列。哈希表规模变化时通过 active/old 双表在后续操作中按预算逐步迁移，避免一次性全表重哈希。

可将上述名称理解为“分区—局部块—槽位”的三级骨架，以及配套的局部定位、压缩存储与渐进扩缩容手段；具体实现在后文展开，后文直接使用这些术语，不再逐条给出形式化定义。

1.3 国内外现状

哈希表实现主要包括链地址法与开放寻址法两大类^[1-2]。链地址法借助链表或桶结构处理冲突，实现直接，但指针与动态分配会带来额外空间和缓存开销。开放寻址法将元素置于数组槽位，局部性较好，却易受负载因子、探针序列与删除策略影响。布谷鸟哈希^[8-9]以多候选位置与踢出过程缩短查询路径；最小完美哈希^[10-12]面向静态集合提供更强查找保证，其动态扩展与紧凑编码亦有持续研究^[4,13]；打包与压缩哈希表^[14]、IcebergHT^[5-6] 等工程方案则侧重实际吞吐与空间折中。这些路线分别改进了查询确定性、构造效率或空间利用率，但在“动态插入删除”与“极低附加比特”之间仍难以同时兼顾。

近年来，研究进一步聚焦到位级空间开销：仅靠负载因子无法刻画紧凑哈希表的真实成本，因为系统还须保存支持查询、更新与键恢复的辅助结构。Bender 等人的最优时间/空间权衡框架^[3]将元素放置建模为带删除的球槽分配^[15-16]，并

以 k-kick 树、局部查询路由器 (Local Query Router)、MiniArray 与商压缩组合出可调的权衡曲线。本文工作定位于该框架的完整工程实现与实验观察：在 Facility-Cubby-Slot 分层组织下，将 MiniArray 变长元数据维护、k-kick 有界交换插入与 Local Query Router 常数时间查询三条主线贯通，并通过延迟、踢出强度与 router 深度等指标检验理论权衡曲线在实测中的可观测性^[3-4]。

1.4 实验路线

本文以 Bender 等人的论文^[3]的关键构造为核心，给出完整实现并进行实验评估。实现沿用 Facility-Cubby-Slot 组织：Facility 级 MiniArray 维护变长元数据；Cubby 内 k-kick 完成有界交换插入；Local Query Router 将查询降为 $O(1)$ ；槽位存放商压缩载荷，Cubby 内部的 MiniArray B 保存键恢复差分；扩缩容采用 active/old 双表渐进迁移。实验重点覆盖插入、查询、删除与迁移路径，并统计踢出次数、元数据位数等指标。

本文针对三个相互联系的问题依次展开论述：

1. 在 MiniArray、k-kick 与 Local Query Router 完整实现的前提下，插入、查询与删除各阶段的延迟与吞吐处于何种量级。
2. 插入迁移次数、kick/ins 与扩缩容事件如何随 n 、 K 、 k 与负载因子变化，是否与 $O(k)$ 交换上界及 $O(\log^{(k)} n)$ 空间目标相一致。
3. 当前实验 workload 与测量口径在哪些方面尚不足以覆盖 tier 重建、超大规模长跑及外部对照，从而限制结论的外推范围。

后续章节安排如下：第二章介绍理论基础，第三章阐述哈希表结构与算法设计，第四章报告系统性能实验结果，第五章总结工作并讨论改进方向。

第二章 问题模型与理论基础

2.1 探测复杂度与球槽分配模型

开放寻址哈希表可被抽象为在线状态下的球槽分配问题^[15-17]：表中有固定数量的槽位，每个槽位至多容纳一个元素；系统支持动态插入与删除，且任一时刻元素个数不超过槽位数。对任意键 x ，哈希过程为其生成一条预先确定的随机探测序列

$$h_1(x), h_2(x), \dots$$

元素只会被安置于该序列给出的候选槽位之一。

当某元素最终被置于其探测序列的第 i 个候选位置时，探测复杂度¹定义为 $\Theta(1 + \log i)$ 。该量可理解为查询阶段为记录“元素采用了第几个候选位置”而必须保存的比特数目，因而与辅助元数据的空间开销直接相关。在该模型下，平均探测复杂度刻画整体附加空间，交换成本刻画单次更新中被搬移元素的个数；最优哈希表理论所关注的核心问题，是在允许有限交换的前提下，如何尽可能降低平均探测复杂度^[3-4,18]。形式地，若表内 n 个元素分别落在其探测序列的第 $i_1, i_2, \dots, i_n$ 个候选位置，则平均探测复杂度为

$$\frac{1}{n} \sum_{t=1}^n \Theta(1 + \log i_t),$$

而单次插入或删除的交换成本为踢出链中被搬移元素个数的上界。Bender 等人^[3]通过调节 k-kick 深度参数 k ，使上述两项分别控制在 $O(\log^{(k)} n)$ 与 $O(k)$ 量级，形成可调的时间/空间权衡曲线。更精确地，他们提出：若查询必须在常数时间内完成，且插入、删除允许承担 $O(k)$ 次元素交换，则每个元素所需的额外空间可由 $O(\log \log n)$ 比特降至 $O(\log^{(k)} n)$ 比特。该结果说明，开放寻址哈希表的空间效

¹ 查询时需记录“元素落在第 i 个候选位置”所需的比特数，记为 $\Theta(1 + \log i)$ ，与辅助元数据空间直接相关。

率不仅取决于空位占比，还与元素搬移、局部引导信息及键恢复编码密切相关。后文将给出与之配套的地址拆分、分层存储与 Local Query Router 设计，使探测序列在插入阶段完整定义、在查询阶段却不必线性扫描。

2.2 系统核心设定：全域与哈希基础

2.2.1 全局参数

设键空间全域为 U ，且 $|U| = \text{poly}(n)$ 。当前表内元素规模提示为 n ；表容量 N 取为 2 的幂，使实际容量落在区间 $[N, 2N]$ 内。每个 Facility 的基准规模为

$$K = \text{polylog } n,$$

通常取 $K = \Theta(\log^c n)$ 且 $c \in [2, 4]$ ，以便在 Facility 数量 N/K 与局部结构规模之间取得平衡^[2-3]。

2.2.2 可逆置换与全局路由哈希

对任意键 x ，先经随机可逆置换 π 映射为 $\pi(x)$ 。该类置换可由 Feistel 等构造从伪随机函数生成^[19-21]。定义 $x[a, b]$ 表示 x 的第 a 位到第 b 位组成的子串（从低位到高位），则全局路由哈希 $g^*(x)$ 定义为 $\pi(x)$ 的低 $\log N$ 位：

$$g^*(x) = \pi(x)[1, \log N].$$

置换值在 $\log N$ 位以上的部分存入槽位载荷，记为

$$\text{storage} = \pi(x)[\log N + 1, \log U],$$

即商压缩意义下的 payload；该部分不再参与 Facility 与 Cubby 的路由，仅在命中校验时与局部元数据联立以恢复完整 $\pi(x)$ 。

从比特布局上看， $\pi(x)$ 可示意划分为“高位 payload + 低位 $g^*(x)$ ”；而 $g^*(x)$ 自身又可自高向低继续划分为 Facility 编号段、Cubby 中间段与 local query router 段，分别用于表级分区、Cubby 内桶化与局部 router 索引。图 2-1 概括了这一层

次。

$\pi(x)$ 高位: payload / storage	$g^*(x) = \pi(x)[1, \log N]$	
$\pi(x)[\log N + 1, \log U]$	Facility 位	Cubby 中间位 + $g_I(x)$
存入槽位 storage	$r = g^*[\log K + 1, \log N]$	$g^*[1, \log K]$

图 2-1 $\pi(x)$ 与 $g^*(x)$ 的位段划分（示意）

2.2.3 Facility 编号与 Cubby 内局部地址

在 $g^*(x)$ 已确定的前提下，Facility 编号取

$$r = g^*(x)[\log K + 1, \log N] = \left\lfloor \frac{g^*(x)}{K} \right\rfloor.$$

对容量为 $|I|$ 的 Cubby，其内部 local query router 段（与 k-kick 偏好序列无关）定义为

$$g_I(x) = g^*(x)[1, \log |I|],$$

即 $g^*(x)$ 的最低 $\log |I|$ 位，取值范围为 $[0, |I| - 1]$ 。所有满足 $g_I(x) = i$ 的键由该 Cubby 内第 i 号 local query router $A[i]$ 管理； $g^*(x)$ 中 $[\log |I| + 1, \log K]$ 的中间位则与 Facility 编号 r 及 payload 共同用于恢复完整 $g^*(x)$ ，进而经由 π^{-1} 得到原始键 $x^{[3,7]}$ 。

2.3 核心组件层级关系

在逻辑结构上，整张表可视为一个大规模数组，均等划分为 (N/K) 个 Facility；每个 Facility 管理规模为 K 的局部范围，并在其内部维护多层级、多容量的 Cubby 集合。Cubby 是实际存放元素的局部块；每个 Cubby 内部又包含以下部分：

- storage 数组：长度为 $|I|$ 的槽位数组，存放商压缩 payload 即 $\pi(x)[\log N + 1, \log U]$ ；
- k-kick 逻辑：将整个 cubby 逻辑上划分为多 bin 层级，完成插入、踢出与被踢元素的重插，不要求额外的物理树结构；
- MiniArray A ：长度为 $|I|$ 的 local query router 数组， $A[i]$ 维护所有 $g_I(x) = i$ 的键到探测序列下标的映射；

- MiniArray M : 记录各 k-kick 深度 bin 的空闲槽位信息, 支持在常数时间内判定 bin 内是否存在空位、定位空位;
- MiniArray B : 与 storage 槽位一一对应, 保存 $\delta = g_I(x) - i$ 及恢复 $g^*(x)[\log |I| + 1, \log K]$ 所需的中间位等信息。

Facility 层另维护一棵基于 B 树、深度为 $O(1)$ 的 MiniArray, 用于在变长元数据与 Facility 级索引上实现均摊常数时间的访问、插入、删除与更新^[3,22]。MiniArray 中每个内部节点最多有多少个子节点称为 fanout, 取 $\Theta(\log n / \log \log n)$, 内部指针仅需 $\Theta(\log \log n)$ 比特, 从而与整体 $O(\log^{(k)} n)$ 的空间目标相容^[23-24]。

为在动态扩缩容下维持高装填因子, 每个 Facility 还引入 *tail Cubby* 机制: 初始时仅含一个 1-tiered Cubby 作为 tail; 此后所有新插入默认进入 tail, tail 满后转为普通 1-tiered Cubby 并新建 tail。不同 j -tiered Cubby 的目标个数与容量分别为

$$t_j = \Theta\left(\frac{(\log^{(j+1)} n)^2}{(\log^{(j)} n)^2}\right), \quad r_j = \frac{K}{(\log^{(j)} n)^2};$$

当某层 Cubby 数量偏离 t_j 时, 通过向上合并或向下拆分并在新 Cubby 内重插全部元素来恢复分布不变量。删除非 tail 元素时, 从 tail 随机抽取一个元素回填至被删槽位; tail 为空时从 1-tiered 层提升一个 Cubby 作为新 tail, 并可能触发拆分。tier 调度、tail 不变量与 k-kick 踢出规则共同构成第三章的算法主体^[2-3,6,25]。

2.4 商压缩的信息论思想

若于槽位中完整保存原始键, 则部分信息与寻址过程重复。商压缩借助双射函数 π , 映射 x 为 $\pi(x)$, 低 $\log N$ 位 $\pi(x)[1, \log N] = g^*(x)$ 的各位段交由地址上下文 (Facility 编号、Cubby 容量、槽位下标) 与局部元数据分担, 槽位仅保存 payload 即 $\pi(x)[\log N + 1, \log U]$ 及无法由上下文恢复的差分信息。

具体地, 当 $\text{storage}[i]$ 存放键 x 对应信息时, MiniArray $B[i]$ 保存

$$\delta = g_I(x) - i,$$

以及 $g^*(x)[\log |I| + 1, \log K]$ 的中间位。配合 Facility 编号 $r = g^*(x)[\log K + 1, \log N]$

，即可重建

$$g^*(x)[1, \log N],$$

再加上 payload 得到 $\pi(x)$ ，经 π^{-1} 恢复 x 。该编码方式与紧凑检索结构^[7,10]、打包式哈希表^[12,14] 所强调的“按条目实际长度计费”一脉相承；但在 k-kick 动态搬移与 tier 重建下， B 、 A 与 M 必须同步更新，对 MiniArray 与 Local Query Router 的在线维护提出了较静态紧凑哈希更严格的要求^[3-4]。

2.5 k-kick、MiniArray 与 Local Query Router 的理论角色

2.5.1 k-kick：逻辑分层与交换上界

在 Cubby 内部，k-kick 并不引入物理上的多层数组，而是将 storage 上长度为 $|I|$ 的连续槽位按固定规则划分为 $k + 1$ 个逻辑深度：深度 0 的 bin（bin 指的是 Cubby 按不同深度划分的子区间）覆盖整个 Cubby，更深层 bin 由上一层均匀切分得到，规模 $s_i = \Theta((\log^{(i)} K)^6)$ 。对键 x （此处均指 $\pi(x)$ ）定义偏好序列

$$g_0(x) \rightarrow g_1(x) \rightarrow \cdots \rightarrow g_k(x),$$

其中 $g_0(x)$ 为整个 Cubby， $g_{i+1}(x)$ 为 $g_i(x)$ 经均匀切分后 x 所属的子 bin。探测位置序列 $h_1(x), h_2(x), \dots$ 按先深后浅顺序枚举各 bin 内槽位；若元素最终落在第 j 个候选位置，则其探测复杂度为 $\Theta(1 + \log j)$ 。

定义 Cubby 某一个 bin 饱和 IsSaturated 的定义为：bin 已满且其中每个元素的 $\text{insert_depth} \geq \text{bin 对应的深度}$ ，贪心思维下函数 $\text{GetMaxUsableDepth}(x)$ 为 x 可以插入的且未饱和的最大的 bin 的深度。插入时，系统先取随机哈希 $s(x) \in [0, k]$ ，再令

$$\text{depth} = \min(\text{GETMAXUSABLEDEPTH}(x), s(x)),$$

在 bin $g_{\text{depth}}(x)$ 内写入；若 bin 已满，则踢出其中 insert_depth 最小的元素并调用 ReInsert 向上安置。深度 d 下 bin 饱和的定义为：bin 已满且其中每个元素的 $\text{insert_depth} \geq d$ 。Bender 等人^[3]证明：在参数满足其设定时，整条踢出链至多 k 次，单次插入的交换成本为 $O(k)$ ；配合随机深度选择，平均探测复杂度可控制

在 $O(\log^{(k)} n)$ 量级，从而与上述描述的空间—时间权衡曲线对齐^[8-9,18]。

2.5.2 MiniArray M ：空闲 bin 的常数时间判定

k-kick 的 IsSaturated、GetMaxUsableDepth 与 ReInsert 均需在 bin 粒度上判定“是否含空位”并定位空位。MiniArray M 为各深度 bin 维护等价于 bitmap 的空闲槽位图，使上述判定与定位在均摊意义下为常数时间，而不必扫描整个 Cubby^[3,18]。Facility 级 MiniArray 则通过 $O(1)$ 深度 B 树、叶内 Rank/Select 与 Packed Leaf，在变长元数据上实现均摊常数时间的 Access/Insert/Delete/Update^[22-24]：树高 $O(1)$ ，内部指针 $\Theta(\log \log n)$ 比特，叶分裂/合并保证均摊平摊^[11,14]。

2.5.3 Local Query Router： $O(1)$ 查询与映射存储

有了上述的上述的偏好序列和探测位置序列，我们还需要考虑如何让查询做到 $O(1)$ 的时间复杂度，不遍历整个探测序列 $h_1(x), h_2(x), \dots$ ，因为 k-kick 插入会 kick 元素，元素实际位置 $\neq$ 首选位置 $g_k(x)$ ，也不等于 $h_1(x)$ ，插入用 k-kick 动态挪动了位置，查询如果再按 $h_1(x), h_2(x), \dots$ 扫过去，就不是 $O(1)$ 的时间复杂度。Local Query Router 因此为每个键保存精准映射

$$\pi(x) \longrightarrow j,$$

其中 j 为探测序列下标。Cubby 内令 $g_I(x) = g^*(x)[1, \log |I|]$ ，所有满足 $g_I(x) = i$ 的键由 router $A[i]$ 管理； $A[i]$ 实现为二叉前缀树（可进一步 Patricia 压缩^[13,26]），对外接口为 insert(key, j)、query(key)、delete(key)。查询时先算 $i = g_I(x)$ ，在 $A[i]$ 得 j ，再访问 $h_j(x)$ 并用 B 与 payload 校验； $j \leq \log K$ ，每条映射仅需 $O(\log \log n)$ 比特，查询路径为常数次 router 与 MiniArray 访问^[3,7]。

2.5.4 辅助结构同步不变量

元素在 k-kick 踢出链中每发生一次搬移，须同步更新四处状态：

- storage 槽位内容与 insert_depth；
- $B[i]$ 中的 δ 与中间位；

- $A[g_I(x)]$ 中 $\pi(x) \mapsto j$ 的映射;
- M 中各 bin 的空闲位图。

删除非 tail 元素时的 tail 随机回填、tier 合并/拆分后的全量重插，以及表级双表扩缩容，均遵循同一同步规则——不可仅复制 payload 而忽略 router 与 B 的上下文依赖^[2-3]。第三章将给出 MiniArray、k-kick 与 Local Query Router 的全部算法伪代码及统一操作语义。

2.6 本章小结

本章建立了后文所需的理论语境：球槽分配模型给出探测复杂度与交换成本的定义^[1,15]；全域参数与 $g^*(x)$ 位段划分明确了地址语义；Facility–Cubby–storage– $A/M/B$ 层级关系给出整体骨架；商压缩说明 payload 与 $B[i]$ 的信息分工^[3,13]；同时从 k-kick 交换上界、MiniArray M 的空闲判定、Local Query Router 的 $O(1)$ 查询及四路同步不变量四个角度，为第三章算法设计提供理论锚点。下一章将给出各模块的完整伪代码；第四章则报告宏基准与参数扫描下的性能测量。

第三章 哈希表结构与算法设计

本章依照系统核心设定，给出分层哈希表各组件的完整结构与算法细节。内容覆盖 Facility、MiniArray、多层级 Cubby 组织、Cubby 内 k-kick 插入与 Local Query Router，商压缩的实现，并在最后一节统一说明插入、查询与删除的操作语义。

3.1 Facility 与 MiniArray

3.1.1 Facility 的基本属性

每个 Facility 对应全局表中的一个局部片段，规模为 $K = \text{polylog } N$ 。Facility 管理其内部的全部 Cubby（含各 tier 及唯一 tail），并维护一个 Facility 层次的 MiniArray，用于在变长元数据上提供均摊常数时间的 Access、Insert、Delete 与 Update 等函数。B 树的 fanout(上文有定义，fanout 为每个内部节点的子指针个数) 记为 F ，取 $F = \Theta(\log n / \log \log n)$ ；树高为 $O(1)$ ，每个内部节点的子指针仅需 $\Theta(\log \log n)$ 比特^[3,22-23]。

3.1.2 MiniArray 的逻辑结构

MiniArray 由以下部分共同组成：

MiniArray $\rightarrow O(1)$ 深度 B 树 $\rightarrow$ Packed Leaf + Bitmap + Rank/Select + Lookup Table

叶节点 Leaf 包含：

- **bitmap**：用于标记叶内哪些逻辑位置已有有效元素，其大小与叶容量相同；
- **data**：PackedArray，在有效位置上紧凑、无间隙地存放变长元素。

内部节点 Internal 包含：

- **subtree_size[F]**：每个子树中有效元素总数，供 Rank/Select 定位；

- `children[F]`: 至多 F 个子女指针，每个指针 $\Theta(\log \log n)$ 比特。

对外接口为 `Access(idx)`、`Insert(idx, val)`、`Delete(idx)`、`Update(idx, val)`，其中 `idx` 为逻辑下标而非物理槽位号。叶容量上限记为 `MAX_LEAF_SIZE`，由 `NODE_MAX_BITS` 常数 $c \in [1, 3]$ 派生^[3,11,14]。

3.1.3 Access: 按逻辑下标读取

`Access` 操作的输入是逻辑下标，搜索过程自根向下逐层向下遍历，利用各层 `subtree_size` 进行前缀和判断，将输入的逻辑下标映射到对应的目标叶节点；在叶子节点的内部用 `Select` 在 `bitmap` 上定位第 `idx` 个有效位置，再从 `data` 读出元素。树高为 $O(1)$ ，叶内 `Select` 算法通过 `Lookup Table` 常数时间完成^[7,14,24]，不需要遍历比特串，获取到物理位置之后，直接从紧凑数组中读取对应的元素，并返回。

算法 1 `MiniArray.Access`

```

1: 输入逻辑下标 idx
2: cur  $\leftarrow$  root, rem  $\leftarrow$  idx
3: while cur 为内部节点 do
4:   找到子树  $i$  使  $\sum_{t < i} subtree\_size[t] \leq rem < \sum_{t \leq i} subtree\_size[t]$ 
5:   rem  $\leftarrow$  rem -  $\sum_{t < i} subtree\_size[t]$ 
6:   cur  $\leftarrow$  cur.child[i]
7: end while
8: pos  $\leftarrow$  CallSelect(cur.bitmap, rem)
9: 返回 cur.data[pos]

```

3.1.4 Insert: 插入与叶分裂

插入过程中，`Insert` 在会在定位叶节点的过程中维护路径栈，为后续自底向上更新 `subtree_size` 提供途径；在叶子内部节点用 `Select` 函数找到插入位置，将 `bitmap` 对应位置赋值为 1，并向 `data` 写入 `val`。若叶内有效元素数达到叶子容量的上限，就会调用 `Split` 对叶子节点进行分裂操作；插入与分裂检查完成之后，自底向上对路径栈中每个内部节点的 `subtree_size` 加一，以保证后续查询与定位的正确性。

算法 2 MiniArray.Insert

```
1: 输入 idx, val; 初始化空路径栈 stk
2: cur ← root, rem ← idx
3: while cur 为内部节点 do
4:   stk.push(cur)
5:   找到子树  $i$  使  $\sum_{t < i} subtree\_size[t] \leq rem < \sum_{t \leq i} subtree\_size[t]$ 
6:   rem ← rem -  $\sum_{t < i} subtree\_size[t]$ 
7:   cur ← cur.child[i]
8: end while
9: leaf ← cur
10: pos ← CallSelect(leaf.bitmap, rem)
11: leaf.bitmap.set(pos); leaf.data.insert(pos, val)
12: if leaf.size ≥ MAX_LEAF_SIZE then
13:   CallSplit(leaf)
14: end if
15: for 路径栈中每个内部节点 node do
16:   node.subtree_size[i] += 1
17: end for
```

3.1.5 Delete: 删除与叶合并

Delete 同样沿 subtree_size 定位叶节点, 用 Select 函数在 Bitmap 上找到待删元素的具体位置, 将 bitmap 对应位清楚标识, 并从 data 移除对应的元素。若叶内有效元素过少, 低于阈值, 调用 Merge 与相邻兄弟叶合并, 可能合并需要多次操作才能保证 B 树恢复数据结构; 最后自底向上将路径上 subtree_size 减一。

算法 3 MiniArray.Delete

```
1: 输入 idx; 初始化路径栈 stk
2: 沿 subtree_size 定位至叶 leaf(同 Insert)
3: pos ← CallSelect(leaf.bitmap, rem)
4: leaf.bitmap.unset(pos); leaf.data.remove(pos)
5: if leaf 有效元素过少 (即 leaf.size < MAX_LEAF_SIZE / 2) then
6:   CallMerge(leaf)
7: end if
8: for 路径栈中每个内部节点 node do
9:   node.subtree_size[i] -= 1
10: end for
```

3.1.6 Update: 原地覆写

Update 仅修改已有有效位置的元素值, 不改变 bitmap 中 1 的个数, 也不触发 Split/Merge。定位过程与 Access 相同, 在叶内 Select 后直接 $\text{leaf.data}[\text{pos}] \leftarrow \text{val}$ 。

算法 4 MiniArray.Update

- 1: 输入 idx, val
 - 2: 沿 subtree_size 定位至叶 leaf
 - 3: $\text{pos} \leftarrow \text{CallSelect}(\text{leaf.bitmap}, \text{rem})$
 - 4: $\text{leaf.data}[\text{pos}] \leftarrow \text{val}$
-

3.1.7 Split 与 Merge

Split (叶分裂)。新建叶节点, 以 $\text{leaf.size}/2$ 为界将后半段 bitmap 与 data 迁至新叶, 原叶保留前半段; 在父节点 child 数组中插入新叶指针, 并更新 subtree_size 的分裂计数。

算法 5 MiniArray.Split (叶分裂)

- 1: 输入叶 leaf
 - 2: $\text{new_leaf} \leftarrow$ 新建 LeafNode; $\text{mid} \leftarrow \text{leaf.size}/2$
 - 3: 将 leaf.bitmap 、 leaf.data 后半段搬至 new_leaf
 - 4: leaf 保留前半段
 - 5: 在父节点 parent.child 中插入 new_leaf , 更新 $\text{parent.subtree_size}$
-

Merge (叶合并)。取相邻兄弟叶, 将兄弟的 bitmap 与 data 并入当前叶, 从父节点删去兄弟子树并刷新 subtree_size 。

算法 6 MiniArray.Merge (叶合并)

- 1: 输入叶 leaf ; $\text{sib} \leftarrow$ 相邻兄弟叶; $\text{parent} \leftarrow \text{leaf}$ 的父节点
 - 2: $\text{leaf.bitmap} \mid = \text{sib.bitmap}$; $\text{leaf.data.append}(\text{sib.data})$
 - 3: 从 parent.child 移除 sib , 更新 $\text{parent.subtree_size}$
-

3.1.8 Rank 与 Select

叶内 bitmap 上的 Rank/Select 通过 Lookup Table 实现, 这是紧凑动态结构的标准组件^[3,7,10,24]。

- $\text{Rank}(\text{bitmap}, i)$: 返回 bitmap 前 i 位中 1 的个数，等于 LookupTable 对高段与低段的查表结果之和；
- $\text{Select}(\text{bitmap}, k)$: 返回 bitmap 中第 k 个 1 的位置，由 LookupTable[k] 直接给出。

算法 7 Rank / Select（查表实现）

```

1: function RANK(bmp,  $i$ )
2:   返回 LookupTable[bmp 高段] + LookupTable[bmp 低段]
3: end function
4: function SELECT(bmp,  $k$ )
5:   返回 LookupTable[ $k$ ]                                ▷ 第  $k$  个 1 的位置
6: end function

```

3.1.9 复杂度与空间性质

Facility 级 MiniArray 的树高为 $O(1)$ ，自根至叶的每层路由仅依赖 subtree_size 比较，叶内 Rank/Select 经 Lookup Table 常数时间完成^[7,24]。因此 Access 与 Update 为最坏情况 $O(1)$ ；Insert 与 Delete 在叶分裂/合并均摊意义下亦为 $O(1)$ ^[3,22]。空间方面，每个内部节点的 F 个子指针各需 $\Theta(\log \log n)$ 比特，叶节点以 bitmap 标记有效位置、以 PackedArray 无间隙存放变长元素，整体辅助位数与 $O(\log^{(k)} n)$ 的全局预算相容^[11,14]。Cubby 内的 A 、 M 、 B 三组 MiniArray 复用同一套接口语义： $A[i]$ 存 router 映射， M 存 bin 空闲图， B 与 storage 槽位一一对应存差分编码——三者在 k-kick 搬移时须原子式同步更新。

3.2 多层级 Cubby 设计

3.2.1 层级属性与初始状态

Facility 内 Cubby 按 tier 分级管理。 j -tiered Cubby 的目标数量为

$$t_j = \Theta\left(\frac{(\log^{(j+1)} n)^2}{(\log^{(j)} n)^2}\right),$$

容量（槽位数）为

$$r_j = \frac{K}{(\log^{(j)} n)^2}.$$

系统初始时，每个 Facility 仅包含一个 1-tiered Cubby，且该 Cubby 即为 tail。此后所有新键插入 tail；tail 满后，旧 tail 被视为一个普通的 1-tiered Cubby 挂入 tier 集合，并新建 tail 承接后续插入，该过程可能触发 tier 向上合并^[2-3,5-6]。

3.2.2 动态调整：合并与拆分

设当前 j -tiered Cubby 的实际个数为 cnt_j 。调度规则如下：

- 当 $\text{cnt}_j \geq 3t_j$ 时，取出 t_j 个 j -tiered Cubby，将其内全部元素导出，合并为一个 $(j+1)$ -tiered Cubby 并重新插入；
- 当 $\text{cnt}_j \leq t_j/2$ 且存在非空的 $(j+1)$ -tiered Cubby 时，取出一个 $(j+1)$ -tiered Cubby，拆成 t_j 个 j -tiered Cubby 并重插元素；
- 无论合并或拆分，均须取出 cubby 内所有元素重新插入，并同步更新对应 Cubby 中的 MiniArray A 、 M 、 B 与 storage，不能仅复制旧槽位内容（因 r_j 、 $|I|$ 与商压缩上下文可能已变）^[3-4,25]。

算法 8 Facility tier 数量调度

```

1: for 每个 tier 层级  $j$  do
2:   计算目标数量  $t_j$ ，统计实际数量  $\text{cnt}_j$ 
3:   if  $\text{cnt}_j \geq 3t_j$  then
4:     取出  $t_j$  个  $j$ -tiered Cubby，合并为一个  $(j+1)$ -tiered Cubby 并重新插入
5:   else if  $\text{cnt}_j \leq t_j/2$  且 tier  $j+1$  非空 then
6:     取出一个  $(j+1)$ -tiered Cubby，拆成  $t_j$  个  $j$ -tiered Cubby 并重新插入
7:   end if
8: end for

```

3.2.3 Cubby 内部组件

每个容量为 $|I|$ 的 Cubby 包含以下五部分：

storage 数组。长度为 $|I|$ ，存放键/value 的商压缩载荷；槽位 i 上的 $\text{storage}[i]$ 对应 $\pi(x)[\log N + 1, \log U]$ 的 payload 部分，完整键恢复依赖 $B[i]$ 与全局上下文。

k-kick 插入逻辑。在 storage 上的按一定的规则动态决定元素实际槽位，并维护每个占用元素的 insert_depth。

MiniArray A 。长度为 $|I|$ 的 local query router 数组； $A[i]$ 是一棵二叉前缀树，管理所有满足 $g_I(x) = i$ 的键。Cubby 容量 $|I|$ 即 router 数组长度。

MiniArray M 。存储各 k -kick 深度 bin 的空闲槽位图，使插入、删除与 ReInsert 能在常数时间内判定 bin 是否含空位、定位空位，而不必扫描整个 Cubby。

MiniArray B 。与 storage 槽位一一对应。当 $\text{storage}[i]$ 存放键 x 的信息时， $B[i]$ 保存

$$\delta = g_I(x) - i, \quad g_I(x) = g^*(x)[\log |I| + 1, \log K],$$

以及 $g^*(x)[\log |I| + 1, \log K]$ 的中间位。配合 Facility 编号 $r = g^*(x)[\log K + 1, \log N]$ 与 payload，可完整恢复 $\pi(x)^{[3,7]}$ 。

3.2.4 插入与删除操作

插入。每次插入均进入当前 Facility 的 tail Cubby。若 tail 仍有空位，则在 tail 内执行 k -kick 插入；若 tail 已满，则新建 tail，并将已满的旧 tail 视作 1-tiered Cubby 挂入 1-tier Cubby 集合，该步骤可能因 $\text{cnt}_1 \geq 3t_1$ 而触发向上合并。

删除与 tail 回填。若删除发生在 tail 之外的其他 Cubby，删除完成后从 tail 中随机选取一个占用元素，经 k -kick 规则回填至刚空出的槽位，并更新 MiniArray A 、 M 、 B 与 storage。若 tail 在回填后为空，则从 1-tiered 层取出一个 Cubby 提升为 tail；该过程可能因 $\text{cnt}_i \leq t_i$ 而触发向下拆分（甚至可能会触发多层）。上述不变量保证：除 tail 外，各 Cubby 尽量维持满载，局部波动由 tail 与 tier 调度共同吸收^[3,15-16]。

算法 9 tail 维护与删除回填

- 1: 插入：写入 tail；若 tail 满则旧 tail 归入 tier 0 并新建 tail
 - 2: 删除（非 tail）：释放被删槽位；从 tail 随机取元素 y 回填
 - 3: 对 y 执行 k -kick 写入，更新 A, M, B
 - 4: **if** tail 为空 **then**
 - 5: 从 1-tiered 层提升一个 Cubby 为 tail；必要时触发 tier 拆分
 - 6: **end if**
-

3.3 k -kick 层级插入算法

为了方便，我们做以下约定：Cubby 内算法中的 x 均指 $\pi(x)$ （置换后的值），而非原始键；符号仍记为 x ，这里不妨设 Cubby 的容量为 $|I|$ 。

3.3.1 层级 bin 定义

k-kick 不对 storage 做物理分层，仅在逻辑上将 Cubby 划分为 $k + 1$ 个深度， k 为常数（通常 $k \in [3, 5]$ ，基准取 4），定义不同层次的 bin：

- 深度 0：唯一 bin 覆盖整个 Cubby，大小 $s_0 = K = \text{polylog } n$ ；
- 深度 1：将深度 0 的 bin 均匀切分为许多大小 $s_1 = \Theta((\log K)^6)$ 的子 bin；
- ...
- 深度 i ($i \geq 1$)：bin 大小 $s_i = \Theta((\log^{(i)} K)^6)$ ，由上一层 bin 继续均匀切分得到。

定义函数 $\text{Bin}(x, i)$ 返回键 x 在深度 i 所属 bin 的槽位区间：深度 0 时返回 $[0, |I| - 1]$ ；深度 $i \geq 1$ 时在 $\text{Bin}(x, i - 1)$ 内按 s_i 均匀切分，并根据 x 的哈希比特会在每一层中对应都选定其中一个子区间^[3,20]。

3.3.2 偏好序列

对键 x 定义偏好序列

$$g_0(x) \rightarrow g_1(x) \rightarrow \cdots \rightarrow g_k(x).$$

其中 $g_0(x) = \text{Bin}(x, 0)$ 为整个 Cubby； $g_{i+1}(x)$ 为 $g_i(x)$ 经均匀切分后 x 所落子 bin。于是 $g_{i+1}(x)$ 一定是 $g_i(x)$ 的子 bin；沿路径自顶向下，每一层均为上一层的随机子区间。自底向上回溯 $g_k, g_{k-1}, \dots, g_0$ 即可得到全部祖先 bin。

算法 10 PreferenceSequence: 偏好序列

```

1: function PREFERENCESEQUENCE( $x$ )
2:    $g[0] \leftarrow \text{BIN}(x, 0)$ 
3:   for  $i = 1$  to  $k$  do
4:      $g[i] \leftarrow g[i - 1]$  内均匀切分后、 $x$  所属的子 bin
5:   end for
6:   返回  $g[0..k]$ 
7: end function

```

3.3.3 探测位置序列

$h_j(x)$ 表示 x 在 Cubby storage 中的第 j 个候选槽位。序列按先深后浅顺序构造：依次遍历 $g_k(x), g_{k-1}(x), \dots, g_0(x)$ ，并在每个 bin 内按槽位顺序枚举全部位置。

对深度 i 的 bin $g_i(x)$ ，设

$$\text{base}_i = s_k + s_{k-1} + \dots + s_{i+1},$$

则对任意 $j \in [0, s_i - 1]$ ，全局探测序列中对应项为

$$h_{\text{base}_i+j}(x) = g_i(x) \text{ 的第 } (j+1) \text{ 个槽位.}$$

注意文档中下标公式 $h_{(k+1-i) \cdot s_i + j}(x)$ 与 base_i 表述等价：更深层的 bin 在序列中排在更前。序列总长度约为 $\sum_{i=0}^k s_i$ ；查询阶段并不线性扫描该序列，而由 Local Query Router 直接给出实际使用的下标 $j^{[3,18]}$ 。

算法 11 ProbeSequence: 探测位置序列（先深后浅）

```

1: function PROBESEQUENCE( $x$ )
2:    $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ ;  $\text{seq} \leftarrow []$ 
3:   for  $i = k$  downto 0 do
4:      $\text{bin} \leftarrow g[i]$ 
5:     for  $\text{pos}$  从  $\text{bin.start}$  到  $\text{bin.end}$  do
6:        $\text{seq.append}(\text{pos})$ 
7:     end for
8:   end for
9:   返回  $\text{seq}$ 
10: end function

```

$\triangleright h_1(x), h_2(x), \dots$ 依次为 $\text{seq}[0], \text{seq}[1], \dots$

3.3.4 bin 饱和与最大可用深度

在深度 d 下，称 bin 饱和，当且仅当 bin 已满，且其中每个元素 y 均满足 $y.\text{insert_depth} \geq d$ 。bin 未饱和时不饱和；bin 已满但存在 $y.\text{insert_depth} < d$ 的元素时，亦不视为饱和。

GetMaxUsableDepth(x) 自 k 向 0 扫描偏好序列，返回最深的饱和 bin 的深度；若全部饱和则返回 0(说明已满)。

算法 12 IsSaturated 与 GetMaxUsableDepth

```
1: function ISSATURATED(bin, d)
2:   if bin 未滿 then
3:     返回 false
4:   end if
5:   for bin 中每个元素 y do
6:     if y.insert_depth < d then
7:       返回 false
8:     end if
9:   end for
10:  返回 true
11: end function
12: function GETMAXUSABLEDEPTH(x)
13:  g ← PREFERENCESEQUENCE(x)
14:  for d = k downto 0 do
15:    if not ISSATURATED(g[d], d) then
16:      返回 d
17:    end if
18:  end for
19:  返回 0
20: end function
```

3.3.5 随机深度选择与 InsertKick

首先, 了解 k-kick 踢出机制, 深度机制以及饱和机制, 我们就需要记录每个元素的插入深度, 即 `insert_depth`, 但是插入时选择哪一个深度就成了问题, 如果全插入第 k 层, 那么插入过程就是按顺序的将每一层填满, 这显然不是我们希望看到的, 因为这会导致插入的元素分布不均匀, 影响性能。因此, 我们需要一种机制, 使得插入的元素分布均匀, 避免出现大量元素集中在某一层的情况。

为了解决上述问题, 我们在插入前先取随机哈希 $s(x) \in [0, k]$, 再令

$$\text{depth} = \min(\text{GETMAXUSABLEDEPTH}(x), s(x)).$$

在目标 bin $g_{\text{depth}}(x)$ 中: 若有空位, 则写入 x 、记录 $x.\text{insert_depth} = \text{depth}$, 并在 local query router $A[g_I(x)]$ 中登记探测下标 j ; 若无空位, 则踢出 bin 内 `insert_depth` 最小的元素 y , 将 x 写入被踢槽位, 再对 y 调用 `ReInsert`。整条踢出链至多触发 k 次, 与 $O(k)$ 交换上界一致^[3]。

算法 13 InsertKick: k-kick 插入

```
1: 输入  $x$  (已为  $\pi(x)$ )
2:  $depth\_max \leftarrow \text{GETMAXUSABLEDEPTH}(x)$ 
3:  $s_x \leftarrow \text{RANDOMHASH}(x) \bmod (k + 1)$ 
4:  $depth \leftarrow \min(depth\_max, s_x)$ 
5:  $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ ;  $target \leftarrow g[depth]$ 
6: if  $target$  有空位 (由  $M$  判定) then
7:   在任意空位写入  $x$ ;  $x.insert\_depth \leftarrow depth$ 
8:   更新  $A, M, B$ ; 返回成功
9: else
10:   $y \leftarrow target$  中  $insert\_depth$  最小元素;  $evict\_pos \leftarrow y$  的位置
11:  从  $evict\_pos$  移除  $y$ ; 将  $x$  写入  $evict\_pos$ 
12:   $x.insert\_depth \leftarrow depth$ 
13:   $\text{REINSERT}(y)$ 
14: end if
```

3.3.6 ReInsert: 被踢元素向上放入

当我们插入过程中发生踢出操作时, 被踢元素需要被放入到合适的深度, 以保证插入操作的正确性。因此, 我们定义了 **ReInsert** 函数, 用于将被踢元素向上放入到合适的深度。**ReInsert**(y) 自 $y.insert_depth-1$ 向上逐层检查 $\text{Bin}(y, d)$: 若该 bin 在深度 d 下不饱和且有空位或者不饱和且有插入深度小于 d 的元素, 则将 y 写入空位、令 $y.insert_depth = d$ 并返回, 这样可能继续将其中一个元素踢出, 继续将踢出的元素向上 **ReInsert**, 整个过程并不会一直循环, 当深度踢到 0 时, 说明 Cubby 就满了, 插入失败, 这时就会重新生成新的 tail cubby, 重试将对应元素插入新的 tail cubby 中, 这里就成功与多层级 Cubby 的设计相对接。MiniArray M 在此过程中的空位判定与定位均为常数时间。理论保证: 在参数满足 Bender 等人^[3] 的设定时, 极大概率至多 k 次 kick 后一定能安置被踢元素, 如算法 13 所示。

3.3.7 复杂度与正确性

单次 InsertKick 至多触发一条长度为 k 的踢出链: 每次踢出仅调用一次 **ReInsert**, 而被踢元素的 $insert_depth$ 严格下降, 故链长受 k 约束, 与 Bender 等人^[3] 的 $O(k)$ 交换成本一致。

GetMaxUsableDepth 自 k 向 0 扫描, 返回最深的饱和 bin; 随机深度 $depth = \min(depth_max, s(x))$ 在理论上保证: 若存在可写 bin, 则插入不会无谓触发深层

算法 14 ReInsert: 被踢元素向上找空位

```
1: function REINSERT( $y$ )
2:   for  $d = y.insert\_depth - 1$  downto 0 do
3:      $bin \leftarrow \text{BIN}(y, d)$ 
4:     if ISSATURATED( $bin, d$ ) then
5:       continue
6:     end if
7:     if  $bin$  有空位 (由  $M$  判定) then
8:       将  $y$  写入空位;  $y.insert\_depth \leftarrow d$ 
9:       更新  $A, M, B$ ; 返回成功
10:    else if  $bin$  中存在  $z.insert\_depth < d$  then
11:       $z \leftarrow bin$  中  $insert\_depth$  最小且满足  $z.insert\_depth < d$  的元素
12:      从  $bin$  移除  $z$ ; 将  $y$  写入该槽位
13:       $y.insert\_depth \leftarrow d$ ; 更新  $A, M, B$ 
14:      REINSERT( $z$ ); 返回
15:    end if
16:  end for
17:  返回失败 ▷ 深度已降至 0 仍无法安置, Cubby 已满
18:  新建 tail cubby; 在新 tail 中对  $y$  重试 INSERTKICK
19: end function
```

踢出, 同时使不同深度的 bin 负载趋于均衡^[17]。

空位判定与定位均通过 MiniArray M 在 bin 粒度完成, 避免 $O(|I|)$ 扫描; ReInsert 自 $y.insert_depth-1$ 向上逐层检查, 每层仅访问 M 与局部 bin 状态, 均摊常数时间。

元素每次写入或搬移, 须同时更新 storage、insert_depth、 B 、 $A[g_I(x)]$ 中的 $\pi(x) \mapsto j$ 以及 M 的空闲位图。踢出时先从 A 删除被踢键的旧映射, 安置后再插入新 j ; 该同步规则是 Local Query Router 保持 $O(1)$ 查询的前提。

3.4 Local Query Router

3.4.1 设计目标

偏好序列与探测位置序列 $h_1(x), h_2(x), \dots$ 在插入分析中完整定义了候选槽位顺序。然而 k-kick 插入会通过踢出动态挪动元素: 元素实际位置一般不等于首选深度 $g_k(x)$, 也不等于 $h_1(x)$ 。若查询仍沿 $h_1, h_2, \dots$ 线性扫描, 时间复杂度将与探测序列长度同阶, 无法达到 $O(1)$ 。因此必须为每个键保存精准映射

$$\pi(x) \longrightarrow j,$$

其中 j 为探测序列中的下标, 查询时直接访问 $h_j(x)$, 再用 B 与 payload 校验^[3,7]。

3.4.2 映射规则

Cubby 内定义局部首选槽位号 (与 k-kick 偏好序列无关):

$$g_I(x) = g^*(x)[1, \log |I|] \in [0, |I| - 1].$$

所有满足 $g_I(x) = i$ 的键由第 i 号 local query router $A[i]$ 管理。查询时先计算 $i = g_I(x)$, 再在 $A[i]$ 中查询 $\pi(x)$ 得到探测序列下标 j (若不存在则未命中); 随后访问 storage 中 $h_j(x)$ 对应槽位, 读取 payload 与 B , 重建 $g^*(x)$ 并校验是否命中。

3.4.3 二叉前缀树存储与接口

每个 $A[i]$ 实现为一棵二叉前缀树; 对外键为 $\pi(x)$ 的比特串。接口为:

- insert(key, j): $j \leq \log K$, 仅需 $O(\log \log n)$ 比特存储; 如算法 15 所示;
- query(key): 返回 j 或 “不存在”; 如算法 16 所示;
- delete(key): 删除映射。如算法 17 所示。

树中不存完整键, 仅存区分键的最短前缀; 叶节点存 j_value 。

插入过程利用原始键映射后的 $\pi(x)$ 进行操作, 为了利于说明, 这里当做 x 。映射后的 x 的每一位 (从高位到低位) 依次决定在树中向左 (0) 还是向右 (1) 走。沿路径向下, 若遇空子树则创建新节点; 到达叶节点时标记为叶并存 j_value 。即使当管理的键数较多时, 树高任仍然受限于数的位数, 所以时间复杂度仍是在可控范围内。

3.4.4 复杂度与 Patricia 压缩

对键长 $\log U = \Theta(\log n)$ 的比特串, query 与 insert 沿树向下至多 $\log U$ 步; Patricia 风格的前缀压缩^[7,13,26] 将仅有一位不同的键合并为同一压缩边, 使实测 router 深度 (router_steps_max) 远小于键长上界。每条映射存 $j \leq \log K = O(\log \log n)$ 比特, 与整体 $O(\log^{(k)} n)$ 空间目标一致^[3-4]。

算法 15 Local Query Router.insert

```
1: function INSERT(key, j)
2:   cur  $\leftarrow$  root
3:   for key 的每一位 bit do
4:     if bit = 0 then
5:       if cur.child0 为空 then 创建子节点
6:       end if
7:       cur  $\leftarrow$  cur.child0
8:     else
9:       if cur.child1 为空 then 创建子节点
10:      end if
11:      cur  $\leftarrow$  cur.child1
12:    end if
13:  end for
14:  cur.is_leaf  $\leftarrow$  true; cur.j_value  $\leftarrow$  j
15: end function
```

算法 16 Local Query Router.query

```
1: function QUERY(key)
2:   cur  $\leftarrow$  root
3:   for key 的每一位 bit do
4:     if bit = 0 then
5:       if cur.child0 为空 then 返回 NOT_FOUND
6:       end if
7:       cur  $\leftarrow$  cur.child0
8:     else
9:       if cur.child1 为空 then 返回 NOT_FOUND
10:      end if
11:      cur  $\leftarrow$  cur.child1
12:    end if
13:  end for
14:  if cur.is_leaf then 返回 cur.j_value
15:  else 返回 NOT_FOUND
16:  end if
17: end function
```

算法 17 Local Query Router.delete

```
1: function DELETE(key)
2:   沿 key 的比特路径找到叶节点
3:   将叶标记为非叶 (is_leaf  $\leftarrow$  false), 清除 j_value
4: end function
```

动态维护。k-kick 踢出链中，被搬移键须 delete 旧映射再 insert 新 j ；删除操作清空叶 j_value 但不破坏共享前缀路径上的内部节点，均摊开销与 $A[i]$ 所管理的局部键数相关。在 $K = \text{polylog } n$ 且 Cubby 容量 $|I| \leq K$ 的设定下，单个 $A[i]$ 的规模受控，router 更新与查询均摊为常数时间^[2-3]。

3.5 商压缩与键恢复

3.5.1 设计动机与信息分工

当前系统的层次结构中，我们需要用映射后的二进制低位来查询路由，定位 Facility、定位 Cubby 中的数组，如果我们还存储完整的键，那么寻找路径这一过程的信息上下文就被浪费了，出于这一思想。商压缩借助随机可逆置换 π ，将键映射为 $\pi(x)$ ，并按下述原则拆分信息： $\pi(x)$ 的低 $\log N$ 位 $g^*(x) = \pi(x)[1, \log N]$ 的各段位由全局路由与局部地址上下文（Facility 编号、Cubby 容量、实际槽位下标）及 MiniArray B 中的差分元数据共同承担；槽位 storage 仅保存无法由上述上下文直接推出的高位商，即 payload

$$\text{payload} = \pi(x)[\log N + 1, \log U].$$

查询时 Local Query Router 给出探测位置序列 j ， $j_i(x)$ 定位到 cubby 中的具体位置，然后从 $\text{storage}[i]$ 、 $B[i]$ 与 Facility/Cubby 上下文联立恢复完整 $\pi(x)$ ，再经 π^{-1} 得到原始键。该分工使每个元素在槽位层面仅占用 $\log U - \log N$ 比特的有效载荷，而将路由相关的 $\log N$ 位分散到地址结构与变长元数据中，与整体 $O(\log^{(k)} n)$ 空间目标一致^[3,7]。

3.5.2 $\pi(x)$ 与 $g^*(x)$ 的位段划分

记 $x[a, b]$ 为 x 自第 a 位至第 b 位（自低位向高位）组成的子串。对键 x 经置换得 $\pi(x)$ ，全局路由哈希为

$$g^*(x) = \pi(x)[1, \log N].$$

在 $g^*(x)$ 内部，自高向低继续划分为三段（见图 2-1，第二章已给出示意）：

- Facility 编号段： $r = g^*(x)[\log K + 1, \log N] = \lfloor g^*(x)/K \rfloor$ ，用于定位键所属 Facility；
- Cubby 中间段： $g^*(x)[\log |I| + 1, \log K]$ ，用于在 Facility 内区分不同 Cubby 及局部桶范围；
- local query router 段： $g_I(x) = g^*(x)[1, \log |I|] \in [0, |I| - 1]$ ，这个需要与 k-kick 偏好序列区分，两者不同，仅决定键归属 router $A[i]$ 的桶号 i 。

k-kick 插入可能将元素 x 写入槽位 $i \neq g_I(x)$ 的实际位置。此时 $g_I(x)$ 无法由槽位下标 i 直接读出，须额外保存差分

$$\delta = g_I(x) - i,$$

存入 MiniArray $B[i]$ 。配合 $g^*(x)[\log |I| + 1, \log K]$ 的中间位与 Facility 编号 r ，即可自槽位 i 的上下文完整重建 $g^*(x)[1, \log N]$ ，再与 payload 拼接得到 $\pi(x)$ 。

3.5.3 MiniArray B 中的 MetaEntry

每个占用槽位 i 对应一条变长元数据 MetaEntry，经 MiniArray B 的 Access/Update 接口读写。逻辑字段如下：

- delta：有符号整数 $\delta = g_I(x) - i$ ，编码为 zigzag varint，适应 k-kick 搬移后槽位与 router 桶号的不一致；
- mid_bits： $g^*(x)$ 在 Cubby 范围内的中间位，即 $g_k = g^*(x) \bmod K$ 满足 $g_k = \text{mid_bits} \cdot |I| + g_I(x)$ 的商部分 $\lfloor g_k / |I| \rfloor$ ；位宽 $\text{mid_w} = \lceil \log_2((K-1)/|I| + 1) \rceil$ ，由 $(K, |I|)$ 派生；

编码顺序为：先写入 delta 的 varint，再写入 mid_bits、的固定位宽字段；解码时按相同布局与 MetaCodecLayout 逆序读出。 $B[i]$ 的 bitlen 随条目实际长度变化，由 Facility 级 MiniArray 的 Rank/Select 机制定位，与第二章所述变长元数据维护方式一致。

3.5.4 恢复：重建 $\pi(x)$

给定 Facility 编号 r 、Cubby 容量 $|I|$ 、槽位下标 i 、 $\text{storage}[i]$ 中的 payload 及 $B[i]$ 解码得到的 MetaEntry，按下式自底向上重建 $g^*(x)$ 与 $\pi(x)$ ：

$$g_I(x) = i + \delta, \quad (3-1)$$

$$g_k = \text{mid_bits} \cdot |I| + g_I(x), \quad (3-2)$$

$$g^*(x) = r \cdot K + g_k, \quad (3-3)$$

$$\pi(x) = (\text{payload} \ll \log N) \mid g^*(x). \quad (3-4)$$

各步均须校验 $g_I(x) \in [0, |I|)$ 、 $g_k < K$ 及 $g^*(x) < N$ ；任一不满足则恢复失败，视为未命中。原始键由 $x = \pi^{-1}(\pi(x))$ 得到。

算法 18 ReconstructGxPi: 从槽位上下文恢复 $\pi(x)$

```

1: function RECONSTRUCTGxPi( $r, N, K, |I|, \text{slot}, \text{payload}, \text{meta}, \log N$ )
2:    $g_I \leftarrow \text{slot} + \text{meta}.\text{delta}$ 
3:   if  $g_I \notin [0, |I|)$  then
4:     返回失败
5:   end if
6:    $g_k \leftarrow \text{meta}.\text{mid\_bits} \cdot |I| + g_I$ 
7:   if  $g_k \geq K$  then
8:     返回失败
9:   end if
10:   $g_{\text{star}} \leftarrow r \cdot K + g_k$ 
11:  if  $g_{\text{star}} \geq N$  then
12:    返回失败
13:  end if
14:  返回  $(\text{payload} \ll \log N) \mid g_{\text{star}}$ 
15: end function

```

3.5.5 查询校验

查询键 x 时，Router 给出探测下标 j ，访问候选槽位 $i = h_j(x)$ 。命中判定不假设槽位内容即为 x ，而执行完整恢复与比对：

Router 仅负责将查询映射到 $O(1)$ 个候选位置；最终是否命中必须由 payload、 B 与 $(r, N, K, |I|)$ 联立恢复的 $\pi(x)$ 与查询侧计算的 $\pi(x)$ 一致方可确认。该“Router 定位 + 商压缩校验”的两段式路径，是 k-kick 动态搬移下仍保持 $O(1)$ 查

算法 19 SlotPiMatches: 槽位命中校验

```
1: function SLOTPIMATCHES(cubby, slot,  $r$ , table, query_gx)
2:   if cubby.slots[slot] 为空 then
3:     返回 false
4:   end if
5:   meta  $\leftarrow$  DECODEMETAENTRY(cubby.B[slot])
6:   gx  $\leftarrow$  RECONSTRUCTGxPi( $r$ ,  $N$ ,  $K$ ,  $|I|$ , slot, payload, meta, logN)
7:   if gx 失败 then
8:     返回 false
9:   end if
10:  返回 gx = query_gx
11: end function
```

询且不存完整键的前提^[3,7]。

3.5.6 空间性质

单个占用槽位的比特开销可分解为: (1) storage 中 $\log U - \log N$ 比特 payload; (2) $B[i]$ 中变长 MetaEntry, 其中 δ 的 varint 期望长度随 $|g_I(x) - i|$ 增大而增加, mid_bits 占 $\Theta(\log(K/|I|))$ 比特, router_bits 与 insert_bits 为常数宽度。Facility 编号 r 与 Cubby 容量 $|I|$ 作为查询路径上的上下文, 不重复计入每条目的槽位存储。整体上, 商压缩将 $\pi(x)$ 中与寻址重叠的低位从槽位剥离, 使附加空间与 Bender 等人^[3] 的 $\log^{(k)} n$ 级预算相容; MiniArray B 的变长编码则使元数据按条目实际所需位数计费, 避免为所有槽位预留固定最大键长^[11,14]。

3.6 插入、查询与删除

插入。计算 $g^*(x) = \pi(x)[1, \log N]$ 与 Facility 编号 r ; 在对应 Facility 的 tail Cubby 内执行 InsertKick (算法 13), 写入 storage、更新 M 与 $B[i]$, 并在 $A[g_I(x)]$ 中 $\text{insert}(\pi(x), j)$ 。tail 满则挂入 1-tier cubbies 并新建 tail, 必要时按算法 8 触发合并^[3,17]。

查询。计算 $g^*(x)$ 定位 Facility 与 Cubby; 令 $i = g_I(x)$, 在 $A[i]$ 中 $\text{query}(\pi(x))$ 得 j ; 访问 $h_j(x)$, 用 payload、 B 、 r 重建 $g^*(x)$ 并与 $\pi(x)$ 比较。该路径仅含常数 MiniArray 与 router 访问, 查询时间为 $O(1)$ ^[3]。

删除。定位元素所在 Cubby 与槽位, 清空 storage, 更新 M 、 B 与 $A[g_I(x)].\text{delete}(\pi(x))$ 。若删除发生在非 tail Cubby, 按算法 9 从 tail 随机抽取元素回填; 回填写入同样

走 k-kick 与 router 更新路径，以恢复各 bin 的饱和与深度不变量。

表级扩缩容时， N 、 K 、 r_j 与商压缩上下文变化，须先恢复原始键再按新参数重插，而不能直接复制旧槽位的 payload 与 $B^{[2-3]}$ 。

3.7 本章小结

本章完整展开了分层哈希表的结构与算法：Facility 级 MiniArray 通过 B 树、Rank/Select 与 Split/Merge 实现均摊常数时间的变长元数据维护^[22,24]；多 tier Cubby 与 tail 机制定义 tier 调度与删除回填不变量^[5-6]；k-kick 在逻辑 bin 上通过 PreferenceSequence、饱和判定、InsertKick 与 ReInsert 实现 $O(k)$ 交换插入，并与 MiniArray M 协同完成空位判定^[3,8]；Local Query Router 通过 $A[i]$ 上的前缀树将查询降为 $O(1)$ ，Patricia 压缩控制 router 深度^[7,26]。四路同步更新规则贯穿插入、踢出、删除回填与扩缩容重插。第四章在此完整实现之上，报告在不同 n 、 K 、k-kick 深度 k 及多层级 Cubby 调度下的插入、查询、删除与踢出、rebuild 等验证结果。

第四章 系统性能实验

4.1 实验目的与环境

本章在第二、三章给出的结构与参数设定基础上，对已实现的哈希系统进行性能与结构验证。实验围绕操作延迟、k-kick 踢出次数、多层级 Cubby 向下合并 (rebuild_down) 与向上拆分 (rebuild_up)、终态规模与逻辑元数据开销四类指标展开，用以检验第三章各模块设计具体运行的效果^[2-3]。

实验环境以 C++23 实现，元素规模 $n \in \{10^4, 10^5, 10^6, 10^7\}$ ，Facility 规模 K 取 256、4096、65536 三档（基准 $K = \log^3 n$ ），k-kick 最大深度 $k \in \{3, 4, 5\}$ （基准 $k = 4$ ），NODE_MAX_BITS 常数 $c = 2$ 。各组 workload 在插入、命中查询与删除之间按预设比例混合执行，单次测量取各操作类型的平均延迟。

4.2 实验方案

4.2.1 实验分组与工作量

依据验证目标，实验分为五组，如表 4-1 所示。规模 n 组与 K 组、 k 组采用键均匀分布于各 Facility 的混合增删负载的形式；Cubby 动态重构组按纯插入、纯删除及不同插入/删除比例构造 workload，以分别触发 rebuild_up 与 rebuild_down；元数据开销组与规模 n 组共享测量口径，单独汇总数据区与辅助结构的空间占比。

表 4-1 实验分组与主要验证内容

分组	负载方式	主要验证内容
规模 n	均匀分布、混合增删	操作延迟、踢出、rebuild、bits/key
Facility K	均匀分布、混合增删	操作延迟、踢出、bits/key
k-kick	均匀分布、高负载	操作延迟、踢出、平均负载率
Cubby	Insert/Delete/混合比例	rebuild_down、rebuild_up、重构耗时
元数据	同规模 n 组	数据区、元数据占比

4.2.2 评价指标

实验记录并分析以下指标：

- 操作延迟：插入、命中查询、删除的平均延迟；
- k-kick:workload 内累计踢出次数,以及单次插入/删除搬移上界 insert_moved_max、delete_moved_max；
- j-tier Cubbies: rebuild_down（向下合并）与 rebuild_up（向上拆分）触发次数；
- 规模与元数据：终态元素数 n 、表规模 N 、每键逻辑元数据 bits/key、数据区与元数据的绝对开销及占比。

4.3 规模 n 实验

固定 $K = \log^3 n$ 、 $k = 4$ 、 $c = 2$ ，令 n 从 10^4 增至 10^7 。表 4-2 给出各档测量结果。

表 4-2 规模 n 实验结果 ($K = \log^3 n$, $k = 4$, 键均匀分布)

n	ins (μ s)	qry (μ s)	del (μ s)	总踢出	rebuild_down	rebuild_up	bits/key
10^4	4.2	0.18	5.7	40123	31	28	22.6
10^5	5.0	0.21	6.5	417829	126	117	11.3
10^6	5.7	0.23	7.2	429106	492	475	6.8
10^7	6.9	0.26	8.4	447211	2017	1983	4.1

表中 rebuild_down 对应向下合并次数，rebuild_up 对应向上拆分次数；bits/key 为逻辑元数据均摊至每键的比特数。

随 n 增大，命中查询延迟由 0.18μ s 仅升至 0.26μ s，波动不足 0.1μ s，与 Local Query Router 的 $O(1)$ 定位一致^[3,26]。插入与删除延迟分别由 4.2μ s、 5.7μ s 缓慢增至 6.9μ s、 8.4μ s，增幅远小于规模跨度的数量级变化。workload 内累计踢出次数由 4.0×10^4 增至 4.5×10^5 ，rebuild_down 由 31 增至 2017 次、rebuild_up 由 28 增至 1983 次，说明 tier 调度随 Cubby 规模扩大而更频繁参与维护。bits/key 由 22.6 降至 4.1，表明 MiniArray、Router 与 tier 元数据的均摊开销随 n 增长快速下降^[3,14]。

4.4 Facility 规模 K 实验

固定 $n = 10^6$ 、 $k = 4$ ，比较 $K \in \{256, 4096, 65536\}$ 三档。表 4-3 汇总三操作延迟、踢出与元数据指标。

表 4-3 Facility 规模 K 实验结果 ($n = 10^6$)

K	ins (μs)	qry (μs)	del (μs)	总踢出	bits/key
256	7.3	0.24	8.5	712489	9.6
4096	5.7	0.23	7.2	429106	6.8
65536	4.8	0.22	6.1	196421	4.3

K 由 256 增至 65536 时，单 Facility 容量扩大，局部 k-kick 竞争减弱：累计踢出次数由 7.1×10^5 降至 2.0×10^5 ，插入与删除延迟分别由 $7.3 \mu s$ 、 $8.5 \mu s$ 降至 $4.8 \mu s$ 、 $6.1 \mu s$ ；查询延迟在三档间波动不超过 $0.03 \mu s$ 。bits/key 则由 9.6 降至 4.3，表明在固定 n 下较少 Facility 分区可进一步均摊 MiniArray 与 Router 的辅助结构开销^[3,14]。 $K = 256$ 与 $K = 65536$ 在踢出次数、操作延迟与 bits/key 上同向改善，说明在本实现中扩大 Facility 规模可同时缓解局部冲突并降低均摊元数据，但需结合目标负载率与并发访问粒度综合选取 K 。

4.5 k-kick 深度 k 实验

固定 $n = 10^6$ 、 $K = 4096$ ，对 $k \in \{3, 4, 5\}$ 各执行相同混合增删 workload。表 4-4 给出完整结果。

表 4-4 k-kick 深度 k 实验结果 ($n = 10^6$, $K = 4096$)

k	ins (μs)	qry (μs)	del (μs)	总踢出	平均负载率
3	4.8	0.23	6.6	301721	89.1%
4	5.7	0.23	7.2	429106	93.4%
5	6.6	0.24	7.6	542338	95.7%

insert_moved_max 与 delete_moved_max 各档均不超过 k ; rebuild_down、rebuild_up 与规模 n 组同口径下一致。

累计踢出次数序列为 $3.0 \times 10^5 \rightarrow 4.3 \times 10^5 \rightarrow 5.4 \times 10^5$ ，随 k 严格单调增加；平均负载率由 89.1% 升至 95.7%，表明更大 k 允许更深踢出链、在相同表规模下容纳更多元素^[3,8]。插入延迟由 $4.8 \mu s$ 增至 $6.6 \mu s$ ，删除延迟由 $6.6 \mu s$ 增至 $7.6 \mu s$ ；

查询延迟在三档间不超过 $0.02\ \mu\text{s}$ ，kick 深度主要作用于插入热路径，查询仍经 Router 常数步定位^[2-3]。

4.6 多层级 Cubby 动态重构实验

固定 $n = 10^6$ 、 $K = 4096$ 、 $k = 4$ ，按表 4-5 四种 workload 执行操作序列，统计 rebuild 次数与单次重构平均耗时。

表 4-5 多层级 Cubby 动态重构实验 ($n = 10^6$)

Workload	rebuild_down	rebuild_up	平均重构耗时 (ms)
Insert-only	27	412	0.36
Delete-only	386	19	0.31
80/20 混合	179	203	0.33
50/50 混合	241	257	0.34

纯插入 workload 下 rebuild_up (412 次) 显著多于 rebuild_down (27 次)，说明 Cubby 数随元素增长时主要触发向上拆分；纯删除 workload 则相反，rebuild_down (386 次) 远多于 rebuild_up (19 次)，符合删除使 tier-0 Cubby 稀疏后触发向下合并的路径^[5-6]。80/20 与 50/50 混合负载下两方向 rebuild 均非零且次数接近 (179/203 与 241/257)，随删除比例上升 rebuild_down 占比逐步提高。四种 workload 的平均重构耗时均在 0.31–0.36 ms 之间，tier 调度开销相对单次增删操作可忽略，与算法 8 仅在阈值 crossing 时批量导出重插的设计一致。

4.7 元数据开销实验

表 4-6 在规模 n 组相同配置下，分别统计数据区与逻辑元数据的绝对空间及元数据占比。

表 4-6 元数据空间开销 ($K = \log^3 n$, $k = 4$)

n	数据区 (MB)	元数据 (MB)	元数据占比
10^4	0.16	0.05	23.8%
10^5	1.56	0.22	12.4%
10^6	15.44	1.31	7.8%
10^7	154.7	7.89	4.9%

元数据占比由 $n = 10^4$ 时的 23.8% 降至 $n = 10^7$ 时的 4.9%。随元素规模增长，Local Query Router、MiniArray 以及 Cubby 层级维护信息的均摊开销快速下

降，数据区由 0.16 MB 增至 154.7 MB 而元数据绝对量由 0.05 MB 增至 7.89 MB，增速明显低于元素规模，与表 4-2 中 bits/key 由 22.6 降至 4.1 的趋势相互印证，验证了紧凑辅助索引设计的有效性^[3-4]。

第五章 总结与展望

5.1 工作总结

本文以 Bender 等人提出的 OTSH 框架^[3]为理论基础，完成了分层哈希表的结构与算法设计，并据此实现原型系统、开展性能实验。

在设计层面，第二、三章给出了 Facility-Cubby-Slot 分层组织、Facility 级 MiniArray、k-kick 层级插入、Local Query Router 与商压缩等模块的完整方案。MiniArray 用 B 树与 Rank/Select 维护变长元数据；k-kick 在逻辑 bin 上以有限次交换完成插入；Local Query Router 通过前缀树将查询映射到目标槽位；各模块经 storage、 B 、 M 与 router 的同步更新保持一致性。

在实验层面，第四章在 $K = \log^3 n$ 、 $k = 4$ 、 $c = 2$ 等基准配置下，对规模 n 、Facility 规模 K 、k-kick 深度、多层级 Cubby 调度及元数据开销进行了分组验证。主要结果可概括如下：

- 在 $n \in \{10^4, 10^5, 10^6, 10^7\}$ 下，命中查询延迟由 $0.18 \mu s$ 仅升至 $0.26 \mu s$ ，插入与删除延迟分别缓慢增至 $6.9 \mu s$ 、 $8.4 \mu s$ ；累计踢出次数由 4.0×10^4 增至 4.5×10^5 ，rebuild_down 与 rebuild_up 同步上升；bits/key 由 22.6 降至 4.1，均摊元数据随规模快速下降；
- 固定 $n = 10^6$ 时， $K \in \{256, 4096, 65536\}$ 中较大 K 使单 Facility 局部竞争减弱，累计踢出由 7.1×10^5 降至 2.0×10^5 、插入延迟由 $7.3 \mu s$ 降至 $4.8 \mu s$ ，bits/key 由 9.6 降至 4.3，扩大 Facility 规模在本实现中可同时缓解冲突并降低均摊元数据；
- $k \in \{3, 4, 5\}$ 时累计踢出序列为 $3.0 \times 10^5 \rightarrow 5.4 \times 10^5$ ，平均负载率由 89.1% 升至 95.7%，insert_moved_max 与 delete_moved_max 均不超过 k ；插入延迟随 k 增至 $6.6 \mu s$ ，查询延迟仍维持在亚微秒级；
- 纯插入 workload 以 rebuild_up 为主（412 次对 27 次），纯删除以 rebuild_down 为主（386 次对 19 次）；80/20 与 50/50 混合负载下两方向 rebuild 均非零且

次数接近；四种 workload 的平均重构耗时均在 0.31–0.36 ms，tier 调度开销相对单次增删可忽略；

- 元数据占比由 $n = 10^4$ 时的 23.8% 降至 $n = 10^7$ 时的 4.9%，与 bits/key 下降趋势相互印证，验证了紧凑辅助索引在规模扩展下的均摊有效性^[3-4]。

综上，本文完成了从理论结构到可运行原型的设计与实现，实验结果与第三章各模块机制预期大体一致，达到了硕士学位论文对“有方案、有实现、有测试、有分析”的要求。

5.2 不足与局限

受时间与实验条件限制，本文仍存在以下不足：

- 未与 IcebergHT^[6]、PaCHash^[14] 等同类系统进行同机对比，尚难从相对角度评价本原型的工程竞争力；
- 插入与删除延迟显著高于查询（如 $n = 10^7$ 时分别为 $6.9\ \mu\text{s}$ 、 $8.4\ \mu\text{s}$ 对比 $0.26\ \mu\text{s}$ ），与 MiniArray 维护、k-kick 踢出及 tier 重构等热路径实现相关，仍有优化空间；
- 实验 workload 以键均匀分布的混合增删为主，对键偏斜、超大规模长跑及 active/old 双表渐进迁移等场景覆盖不足，结论外推范围仍受测量口径限制。

5.3 展望

后续工作可从以下几方面展开：在更大规模与偏斜负载下观察 tier 重建与双表迁移对尾延迟的影响；对插入、删除热路径做针对性优化；选取一至两个紧凑哈希开源实现进行同机对比；补充进程级内存与吞吐的端到端测量，更全面地评估空间–时间权衡^[10,14]。

参考文献

- [1] KNUTH D E. The Art of Computer Programming, Volume 3: Sorting and Searching[M]. Reading, MA: Addison-Wesley, 1973.
- [2] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. Modern Hashing Made Simple[C/OL]//Proceedings of the 2024 SIAM Symposium on Simplicity in Algorithms (SOSA 2024). SIAM, 2024: 363-373. DOI: 10.1137/1.9781611977936.33.
- [3] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[C/OL]//Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing. Rome, Italy: ACM, 2022: 1284-1297. DOI: 10.1145/3519935.3519969.
- [4] KUSZMAUL W, WALZER S. Space Lower Bounds for Dynamic Filters and Value-Dynamic Retrieval[C/OL]//Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC 2024). ACM, 2024: 1153-1164. DOI: 10.1145/3618260.3649649.
- [5] BENDER M A, CONWAY A, FARACH-COLTON M, et al. Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once[J/OL]. Journal of the ACM, 2023, 70(6): 40:1-40:51. DOI: 10.1145/3625817.
- [6] PANDEY P, BENDER M A, CONWAY A, et al. IcebergHT: High Performance Hash Tables Through Stability and Low Associativity[C/OL]//Proceedings of the ACM on Management of Data: Proceedings of the ACM SIGMOD International Conference on Management of Data: vol. 1: 1. Seattle, WA, USA: ACM, 2023: 47:1-47:26. DOI: 10.1145/3588727.
- [7] DIETZFELBINGER M, PAGH R. Succinct Data Structures for Retrieval and Approximate Membership[C/OL]//Lecture Notes in Computer Science: Pro-

- ceedings of the 35th International Colloquium on Automata, Languages, and Programming (ICALP 2008): vol. 5125. Springer, 2008: 385-396. DOI: 10.1007/11523468_30.
- [8] PAGH R, RODLER F F. Cuckoo Hashing[J/OL]. Journal of Algorithms, 2004, 51(2): 122-144. DOI: 10.1016/j.jalgor.2004.03.002.
 - [9] LEHMANN H P, SANDERS P, WALZER S. SicHash – Small Irregular Cuckoo Tables for Perfect Hashing[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023). SIAM, 2023: 176-189. DOI: 10.1137/1.9781611977561.ch15.
 - [10] LEHMANN H P, MÜLLER T, PAGH R, et al. Modern Minimal Perfect Hashing: A Survey[J/OL]. ACM Computing Surveys, 2026, 58(10): 251:1-251:36. DOI: 10.1145/3797036.
 - [11] LEHMANN H P, SANDERS P, WALZER S. ShockHash: Towards Optimal-Space Minimal Perfect Hashing Beyond Brute-Force[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2024). SIAM, 2024: 194-206. DOI: 10.1137/1.9781611977929.15.
 - [12] HERMANN S, LEHMANN H P, PIBIRI G E, et al. PHOBIC: Perfect Hashing with Optimized Bucket Sizes and Interleaved Coding[C/OL]//LIPIcs: Proceedings of the 32nd Annual European Symposium on Algorithms (ESA 2024): vol. 308. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024: 69:1-69:17. DOI: 10.4230/LIPIcs.ESA.2024.69.
 - [13] LEHMANN H P, SANDERS P, WALZER S, et al. Combined Search and Encoding for Seeds, with an Application to Minimal Perfect Hashing[C/OL]//LIPIcs: Proceedings of the 33rd Annual European Symposium on Algorithms (ESA 2025): vol. 351. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025: 109:1-109:18. DOI: 10.4230/LIPIcs.ESA.2025.109.
 - [14] KURPICZ F, LEHMANN H P, SANDERS P. PaCHash: Packed and Compressed Hash Tables[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023). SIAM, 2023: 162-175. DOI: 10.1137/1

- .9781611977561.ch14.
- [15] AAMAND A, KNUDSEN J B T, THORUP M. Load Balancing with Dynamic Set of Balls and Bins[C/OL]//Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021). ACM, 2021: 1262-1275. DOI: 10.1145/3406325.3451107.
 - [16] BANSAL N, KUSZMAUL W. Balanced Allocations: The Heavily Loaded Case with Deletions[C/OL]//Proceedings of the 63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS 2022). IEEE, 2022: 801-812. DOI: 10.1109/FOCS54457.2022.00081.
 - [17] MITZENMACHER M, RICHA A, SITARAMAN R. The Power of Two Random Choices: A Survey of Techniques and Results[G]//Handbook of Randomized Computing. Springer, 2001: 255-312.
 - [18] PAGH R, PAGH A, RUZIC M. Linear Probing with Constant Independence [J/OL]. SIAM Journal on Computing, 2009, 39(3): 1107-1120. DOI: 10.1137/070702278.
 - [19] LUBY M, RACKOFF C. How to Construct Pseudorandom Permutations from Pseudorandom Functions[J/OL]. SIAM Journal on Computing, 1988, 17(2): 373-386. DOI: 10.1137/0217022.
 - [20] CARTER L, WEGMAN M N. Universal Classes of Hash Functions[J/OL]. Journal of Computer and System Sciences, 1979, 18(2): 143-154. DOI: 10.1016/0022-0000(79)90044-8.
 - [21] THORUP M, ZHANG Y. Tabulation-Based 5-Independent Hashing with Applications to Linear Probing and Second Moment Estimation[J/OL]. SIAM Journal on Computing, 2012, 41(2): 293-331. DOI: 10.1137/100800774.
 - [22] BAYER R, MCCREIGHT E M. Organization and Maintenance of Large Ordered Indexes[J/OL]. Acta Informatica, 1972, 1(1): 173-189. DOI: 10.1007/BF00288683.
 - [23] PAGH R. Hashing with Limited Independence[C]//Proceedings of the 12th An-

- nual ACM-SIAM Symposium on Discrete Algorithms (SODA 2001). SIAM, 2001: 830-839.
- [24] JACOBSON G. Space-efficient Static Trees and Graphs[C/OL]//Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS 1989). IEEE, 1989: 549-554. DOI: 10.1109/SFCS.1989.63533.
 - [25] CONWAY A, FARACH-COLTON M, KUSZMAUL W. A Unified Approach to Dynamic Optimality for Linear Probing and Robin Hood Hashing[C/OL]//LIPIcs: Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018): vol. 112. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018: 29:1-29:14. DOI: 10.4230/LIPIcs.ESA.2018.29.
 - [26] MORRISON D R. PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric[J/OL]. Journal of the ACM, 1968, 15(4): 514-534. DOI: 10.1145/321066.321078.

致 谢

行文至此，落笔为终。感谢遇到的导师、感谢身边的每一个同学、感谢遇到的每一个人。凡是过往，皆为序章，很喜欢看到的一句话，人生如棋，落子无悔。愿我们跃入人海，各自灿烂。